

PHÂN TÍCH & ĐỀ XUẤT CÔNG NGHỆ BACKEND / DATABASE

Dự án: Hệ thống Quản lý Trung tâm Tiếng Anh — Học phần Project 1 (SDP/SDLC)

Người thực hiện: Đoàn Thái Sơn

1. Tiêu chí đánh giá công nghệ Backend

Nhóm đưa ra 6 tiêu chí để so sánh, dựa trên bối cảnh thực tế của dự án và năng lực hiện có của các thành viên:

- Tốc độ phát triển CRUD, vì nghiệp vụ chính của hệ thống là quản lý, chủ yếu là CRUD và một vài phần tính toán nhẹ
- Độ an toàn khi code, tức khả năng phát hiện lỗi sớm ngay lúc biên dịch thay vì lúc chạy thử
- Hệ sinh thái hỗ trợ nghiệp vụ quản lý: xác thực, phân quyền, kiểm tra dữ liệu đầu vào, ORM
- Mức độ đồng bộ với công cụ kiểm thử mà Tester đang dùng (Java, Selenium, TestNG)
- Đường học tập, so với năng lực hiện tại của các thành viên phụ trách
- Khả năng bảo trì và mở rộng trong 9 tuần vận hành hệ thống

2. So sánh các lựa chọn Backend

2.1 Java + Spring Boot

Ưu điểm

- An toàn kiểu dữ liệu cao — phần lớn lỗi được phát hiện ngay lúc biên dịch, giảm đáng kể lỗi phát sinh lúc chạy so với ngôn ngữ động.
- Hệ sinh thái enterprise trưởng thành: Spring Security lo phần phân quyền Admin/User, Spring Data JPA giảm code lặp cho CRUD, Bean Validation cho phép khai báo điều kiện hợp lệ dữ liệu bằng annotation thay vì viết tay.
- Đồng bộ với công cụ kiểm thử hiện tại của Tester (Java, Selenium, TestNG, Gradle) — cùng build tool, cùng hệ sinh thái, thuận lợi khi cần phối hợp đọc hoặc debug code Backend.
- Cấu trúc phân lớp rõ ràng (Controller/Service/Repository), dễ bảo trì và bàn giao trong giai đoạn vận hành.
- JVM chạy ổn định trên nhiều hệ điều hành (macOS/Windows) mà nhóm đang dùng, ít rủi ro lệch môi trường.
- Team đã quen Java từ trước, thuận lợi khi triển khai và hỗ trợ lẫn nhau.

Nhược điểm

- Một luồng CRUD cần viết khá nhiều thành phần (Entity, DTO, Repository, Service, Controller), tốn thời gian hơn so với các stack ít lớp trung gian.
- Người phụ trách Backend chính vẫn đang tìm hiểu thêm về framework, cần thời gian làm quen Spring Boot ở giai đoạn đầu.
- Thời gian biên dịch và khởi động chậm hơn, vòng lặp code — kiểm tra — chạy thử mất nhiều thời gian hơn so với các stack thông dịch trực tiếp.

2.2 Node.js (Express) / Next.js API Routes

Ưu điểm

- Tốc độ phát triển nhanh, ít thành phần trung gian, hỗ trợ tải lại tức thời khi sửa code — phù hợp với thời gian xây dựng ngắn (6 tuần).
- Nếu Frontend cũng dùng React/Next.js, hai bên có thể dùng chung ngôn ngữ và chia sẻ kiểu dữ liệu, giảm sai lệch khi trao đổi dữ liệu.
- Hệ sinh thái CRUD hiện đại và đủ dùng: ORM như Prisma, Drizzle; xác thực như NextAuth, Clerk đều khá phổ biến.
- Cộng đồng lớn, dễ tìm tài liệu tham khảo khi gặp vướng mắc.

Nhược điểm

- Nếu không tuân thủ nghiêm ngặt TypeScript, độ an toàn kiểu dữ liệu thấp, dễ phát sinh lỗi lúc chạy hơn ngôn ngữ tĩnh.
- Không đồng bộ với công cụ kiểm thử hiện tại của Tester (Java) — cần đọc thêm mã nguồn bằng ngôn ngữ khác nếu phải debug sâu ở tầng Backend.
- Một số thư viện có phần biên dịch native (ví dụ bcrypt, sharp) có thể chạy khác nhau giữa macOS và Windows nếu không đóng gói qua container.
- Cấu trúc dự án khá tự do, cần nhóm tự đặt quy ước rõ ràng để tránh code thiếu nhất quán khi nhiều người cùng tham gia.

2.3 Go (Gin / Fiber)

Ưu điểm

- Hiệu năng cao, biên dịch ra file thực thi độc lập, chạy ổn định trên nhiều hệ điều hành.
- An toàn kiểu dữ liệu, mức độ gần tương đương Java.
- Xử lý đồng thời (concurrency) tốt, dù nghiệp vụ quản lý của dự án hiện tại chưa thật sự cần đến điểm mạnh này.

Nhược điểm

- Hệ sinh thái ORM (ví dụ GORM) chưa trưởng thành bằng JPA của Java hay Prisma của Node — phải tự viết thêm nhiều phần mà các stack khác đã hỗ trợ sẵn, như kiểm tra dữ liệu hay quản lý migration.
- Trong nhóm chưa có ai từng dùng Go, sẽ phải học từ đầu — rủi ro không nhỏ khi thời gian xây dựng có hạn.
- Không đồng bộ với công cụ kiểm thử hiện tại của Tester (Java), tương tự Node.js.
- Thế mạnh xử lý đồng thời của Go chưa có nhiều đất dùng với nghiệp vụ quản lý tương đối đơn giản của dự án.

2.4 PHP + Laravel

Ưu điểm

- Tốc độ phát triển CRUD nhanh — Eloquent ORM khá đầy đủ tính năng, Artisan hỗ trợ tạo sẵn model, migration, controller.
- Cơ chế xác thực, phân quyền và một số công cụ quản trị (Laravel Nova, Filament) đã có sẵn, phù hợp với nghiệp vụ dạng quản lý.
- Phổ biến tại Việt Nam, dễ tìm tài liệu tiếng Việt và cộng đồng hỗ trợ.

- Cơ chế quản lý migration rõ ràng, thuận lợi khi cần theo dõi thay đổi cấu trúc database theo thời gian.

Nhược điểm

- Không đồng bộ với công cụ kiểm thử hiện tại của Tester (Java).
- Độ an toàn kiểu dữ liệu thấp hơn Java — PHP 8 có hỗ trợ khai báo kiểu nhưng không nghiêm ngặt bằng ngôn ngữ tĩnh.
- Cần thêm môi trường chạy riêng (PHP-FPM kèm Nginx) khi triển khai, khác với môi trường của Java/Node nếu hạ tầng dùng chung.

3. Bảng tổng hợp đánh giá Backend

Mức đánh giá: Cao — đáp ứng tốt tiêu chí; Trung bình — đáp ứng một phần, có đánh đổi; Thấp — hạn chế rõ, cần cân nhắc kỹ nếu chọn.

Tiêu chí	Java + Spring Boot	Node.js Next.js	/ Go	PHP Laravel +
Tốc độ code CRUD	Trung bình	Cao	Trung bình	Cao
An toàn khi code	Cao	Trung bình (cần TypeScript)	Cao	Thấp
Hệ sinh thái quản lý (auth/ORM/validate)	Cao	Cao	Thấp	Cao
Đồng bộ công cụ kiểm thử (Java)	Cao	Thấp	Thấp	Thấp
Phù hợp năng lực hiện tại của nhóm	Cao	Trung bình	Thấp	Trung bình
Khả năng bảo trì lâu dài (9 tuần)	Cao	Trung bình	Trung bình	Trung bình-Cao

4. So sánh Database: PostgreSQL, MySQL, MongoDB

Hệ thống có nhiều thực thể quan hệ chặt với nhau — Người dùng, Lớp học, Giáo viên, Lịch học, Thanh toán — nên tiêu chí quan trọng nhất khi chọn database là khả năng đảm bảo tính toàn vẹn dữ liệu và xử lý giao dịch chính xác, bên cạnh mức độ phù hợp với hạ tầng và thời gian của nhóm.

4.1 PostgreSQL

Ưu điểm

- Hỗ trợ ràng buộc dữ liệu (constraint), khoá ngoại và giao dịch (transaction) đầy đủ, đúng chuẩn ACID — phù hợp với các nghiệp vụ cần độ chính xác cao như đăng ký lớp học (tránh trùng suất, trùng lịch) hay thanh toán (tránh lệch số liệu khi nhiều người thao tác cùng lúc).
- Cơ chế kiểm soát đồng thời (MVCC) và các mức cô lập giao dịch (isolation level) linh hoạt, giúp xử lý tốt tình huống nhiều người dùng cùng đăng ký, chỉnh sửa dữ liệu một lúc.
- Hỗ trợ kiểu dữ liệu JSON/JSONB, dùng được khi cần lưu một phần dữ liệu có cấu trúc linh động (ví dụ lịch sử thay đổi, cấu hình) mà vẫn giữ được tính quan hệ cho phần dữ liệu còn lại.
- Tích hợp tốt với Spring Data JPA/Hibernate nếu Backend chọn Java, cũng như với Prisma nếu chọn Node.js.
- Miễn phí, mã nguồn mở, chạy ổn định qua Docker ở cả môi trường local lẫn máy chủ, không phát sinh chi phí license.

Nhược điểm

- Cấu hình tối ưu nâng cao (đánh index, tinh chỉnh hiệu năng) phức tạp hơn MySQL một chút, nhưng ở quy mô đồ án hiện tại thì không đáng kể.

- Cần thời gian làm quen ban đầu với một vài khái niệm riêng của PostgreSQL (ví dụ schema, kiểu JSONB) nếu thành viên phụ trách database chưa từng dùng qua.

4.2 MySQL

Ưu điểm

- Phổ biến, tài liệu và cộng đồng hỗ trợ nhiều, dễ tra cứu khi gặp lỗi.
- Cấu hình nhẹ, cài đặt và làm quen ban đầu đơn giản hơn PostgreSQL một chút.
- Tích hợp tốt với hầu hết các ORM phổ biến (JPA, Prisma, Eloquent), không kén stack Backend.

Nhược điểm

- Hỗ trợ ràng buộc dữ liệu phức tạp và kiểu JSON yếu hơn PostgreSQL, đặc biệt khi cần truy vấn sâu vào dữ liệu JSON.
- Ở chế độ mặc định, một số mức cô lập giao dịch dễ gây đọc dữ liệu chưa nhất quán hơn nếu không cấu hình kỹ, trong khi PostgreSQL an toàn hơn ngay từ mặc định.
- Không có ưu thế rõ rệt so với PostgreSQL cho một hệ thống có quan hệ dữ liệu chặt chẽ như hệ thống này, trong khi PostgreSQL cũng miễn phí và không kén stack hơn.

4.3 MongoDB (NoSQL)

Ưu điểm

- Schema linh động, không cần định nghĩa cấu trúc bảng trước — thuận tiện khi cần dựng nhanh một bản thử nghiệm (prototype) hoặc dữ liệu còn thay đổi liên tục.
- Phù hợp với dữ liệu dạng tài liệu, ít quan hệ chằng chịt, ví dụ log thao tác hoặc dữ liệu phân tích.
- Mở rộng theo chiều ngang (horizontal scaling) tốt khi lượng dữ liệu rất lớn — tuy hệ thống hiện tại chưa ở quy mô này.

Nhược điểm

- Không có ràng buộc khoá ngoại và giao dịch đa tài liệu ở mức mạnh như PostgreSQL, nên khó đảm bảo dữ liệu giữa các thực thể (Người dùng, Lớp học, Giáo viên, Lịch học, Thanh toán) luôn khớp nhau.
- Việc liên kết dữ liệu giữa các thực thể phải tự xử lý ở tầng ứng dụng thay vì để database đảm bảo, dễ dẫn đến sai lệch dữ liệu nếu code có sót.
- Khó thống kê, báo cáo theo kiểu quan hệ (ví dụ doanh thu theo lớp, theo giáo viên, theo tháng) vì dữ liệu không được chuẩn hoá thành bảng có quan hệ rõ ràng.
- Không có nhiều lợi thế cho một hệ thống quản lý có nghiệp vụ tương đối ổn định về cấu trúc như trung tâm tiếng Anh.

Tiêu chí	PostgreSQL	MySQL	MongoDB
Toàn vẹn dữ liệu quan hệ / giao dịch	Cao	Trung bình-Cao	Thấp
Phù hợp dữ liệu nhiều thực thể liên kết	Cao	Cao	Thấp
Tích hợp với Spring Data JPA	Cao	Cao	Trung bình
Chi phí / giấy phép	Miễn phí	Miễn phí	Miễn phí (bản Community)
Độ phù hợp với quy mô đồ án	Cao	Trung bình-Cao	Thấp

Kết luận: PostgreSQL là lựa chọn phù hợp nhất cho dự án, vì đáp ứng tốt yêu cầu về quan hệ dữ liệu chặt chẽ và tính chính xác giao dịch của một hệ thống quản lý trung tâm tiếng Anh, trong khi vẫn miễn phí và dễ tích hợp với Backend đã chọn.

5. Đề xuất

Backend: Java + Spring Boot. Đây là lựa chọn phù hợp nhất với bối cảnh hiện tại của nhóm: đồng bộ với công cụ kiểm thử mà Tester đang dùng, cấu trúc rõ ràng nên dễ bảo trì trong 9 tuần vận hành, và có hệ sinh thái phù hợp với nghiệp vụ quản lý. Đây không phải lựa chọn nhanh nhất để viết code, nhưng là lựa chọn giúp kiểm soát rủi ro tốt hơn cho mục tiêu của môn học. Người phụ trách Backend chính sẽ dành thời gian đầu để làm quen framework này, với sự hỗ trợ từ các thành viên khác trong nhóm đã quen Java.

Database: PostgreSQL. Phù hợp với đặc điểm quan hệ dữ liệu chặt chẽ của hệ thống, tích hợp tốt với Spring Data JPA, miễn phí và không có lý do kỹ thuật rõ ràng để chọn phương án khác.

ORM đề xuất: Spring Data JPA + Hibernate, thay vì viết trực tiếp bằng JDBC.

Quản lý thay đổi schema: dùng Flyway hoặc Liquibase để version hoá các thay đổi cấu trúc database, phù hợp với yêu cầu về quản lý cấu hình của môn học.